

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA 0603

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting*
(BL 3)

Assessment Tool Weightage: 20%

QUESTIONS:15

Total Marks:25

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I X. Joshua (192312188) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

X. Joshua

Annexure A

Short Answer Test – Questions and Pseudocode Answers

The following answers are structured for **5 marks each**, covering **concept, application, accuracy, completeness, and clarity**.

1. Design a pseudocode algorithm to read a dataset from a file and preprocess it using array and DataFrame operations. The algorithm should handle missing values, remove duplicate records, normalize numerical columns, and standardize categorical values. Include steps for indexing, filtering invalid entries, and saving the cleaned dataset. Ensure the algorithm clearly defines input, processing steps, and output while maintaining computational efficiency.

Answer:

Input: Dataset file containing numerical and categorical columns.

Output: Cleaned and normalized dataset.

Pseudocode

BEGIN

1. Read the dataset from the input file into a DataFrame.

2. Inspect the dataset:

 Display number of rows and columns.

 Check data types.

 Identify missing values.

3. Remove duplicate records:

 Remove rows that are completely duplicated.

4. Handle missing values:

 FOR each numerical column:

 Replace missing values with column median.

 FOR each categorical column:

 Replace missing values with column mode.

5. Filter invalid numerical entries:

 FOR each numerical column:

 Identify values outside the valid range.

 Remove or replace invalid values.

6. Standardize categorical values:

 Remove leading/trailing spaces.

 Convert categories to a common format.

 Replace inconsistent labels.

7. Normalize numerical columns:

 Find minimum and maximum values.

 Apply:

$$\text{normalized} = (\text{value} - \text{minimum}) /$$

(maximum - minimum)

8. Use indexing to select required rows and columns.

9. Reset the DataFrame index.

10. Validate:

Check that no missing values remain.

Check for duplicate records.

Check numerical ranges.

Check standardized categories.

11. Save the cleaned DataFrame to an output file.

12. Return the cleaned dataset.

END

Efficiency: Vectorized DataFrame and array operations should be preferred over row-by-row loops. Only required columns should be processed to reduce memory usage.

2. Write a detailed pseudocode algorithm to compute statistical measures such as mean, median, standard deviation, and variance from a large dataset. The algorithm should demonstrate how to efficiently use array-based operations, apply filtering to exclude unwanted values, and perform aggregation. Include steps for validating the dataset and handling edge cases such as empty arrays or extreme outliers.

Answer:

Input: Large numerical dataset.

Output: Mean, median, standard deviation, and variance.

Pseudocode

BEGIN

1. Read numerical data into an array.

2. Validate the array:

IF array is empty:

Display "No data available".

STOP.

3. Remove missing and non-numerical values.

4. Filter invalid values according to the required range.

5. Detect extreme outliers:

Calculate mean and standard deviation.

Calculate Z-score for each value.

Identify values with $|Z\text{-score}| > \text{threshold}$.

6. Decide whether outliers should be:

Removed, or

Retained if they are valid observations.

7. Calculate aggregate statistics:
 - MEAN = $\text{sum}(\text{values}) / \text{number_of_values}$
 - MEDIAN = middle value after sorting
 - VARIANCE = average squared deviation from mean
 - STANDARD_DEVIATION = square root of variance
8. Validate results:
 - Check number of valid observations.
 - Check that statistics are finite.
 - Compare minimum and maximum values.
9. Store the statistical results.
10. Display the results.

END

Efficiency: Array-based aggregation functions should be used instead of manually processing every row. For very large datasets, processing can be performed in chunks to reduce memory usage.

3. Develop a pseudocode algorithm to combine two or more DataFrames containing related data. The algorithm must include steps for aligning indices, merging datasets based on a common key, removing duplicates, and validating the integrity of the merged data. Incorporate grouping and sorting operations, and ensure that constraints such as consistency and completeness of records are maintained.

Answer:

Input: Two or more related DataFrames.

Output: Consistent and complete merged DataFrame.

Pseudocode

BEGIN

1. Load all input DataFrames.
2. Inspect:
 - Column names.
 - Data types.
 - Number of records.
 - Missing values.
3. Identify the common key, such as customer_id.
4. Check the common key for duplicates.
5. Standardize data types of the common key.
6. Align DataFrame structures and indexes.
7. Merge DataFrames using the common key.
 - Use INNER join when only matching records are required.

Use OUTER join when all valid records must be preserved.

8. Remove unwanted duplicate records.
9. Group records by the common key.
Identify keys having more than one unexpected record.
10. Sort the merged DataFrame using the common key.
11. Reset the index.
12. Validate:
Check duplicate keys.
Check missing records.
Compare record counts.
Check data types and columns.
13. Generate the final merged DataFrame.

END

Grouping helps detect duplicate keys, while sorting and resetting the index provide a consistent final structure.

4. Create a pseudocode algorithm that filters a dataset based on multiple conditions (e.g., value ranges, categories) and then sorts the filtered results. The algorithm should utilize indexing techniques, logical conditions, and efficient sorting methods. Include steps for handling large datasets and optimizing performance while ensuring that the output meets all filtering constraints.

Answer:

Input: Dataset, filtering conditions, and sorting column.

Output: Filtered and sorted dataset.

Pseudocode

BEGIN

1. Load the dataset into a DataFrame.
2. Identify required columns.
3. Define filtering conditions:
Numerical value must be within valid range.
Category must belong to the required categories.
4. Combine conditions using logical AND/OR.
5. Apply Boolean indexing:
filtered_data = data[
 condition1 AND condition2
]
]
6. Remove invalid or missing records if required.

7. Sort the filtered dataset using the required column.

8. Reset the index.

9. Validate:

Check every numerical value satisfies the range.

Check every category is valid.

Check sorting order.

10. Return the filtered and sorted dataset.

END

For large datasets, Boolean indexing and vectorized operations should be used because they are more efficient than Python loops. Only required columns should be selected to reduce memory consumption.

5. Write a pseudocode algorithm for a complete data analysis workflow that includes reading data from files, converting it into DataFrames, performing grouping and aggregation, and generating plots. The algorithm should specify steps for handling file input/output errors, applying transformations, and producing summarized results suitable for visualization. Ensure clarity in each stage from data loading to final output generation.

Answer:

Input: One or more data files.

Output: Summary tables and visualizations.

Pseudocode

BEGIN

1. Specify input file path.

2. TRY:

Read the file.

IF file cannot be opened:

Display file input error.

STOP.

3. Convert the data into a DataFrame.

4. Inspect:

Columns.

Data types.

Missing values.

Duplicate records.

5. Clean the data:

Handle missing values.

Remove duplicates.

Correct data types.

6. Apply required transformations.
7. Filter records based on analysis conditions.
8. Group data using required categorical columns.
9. Calculate aggregation:
 - SUM
 - AVERAGE
 - COUNT
 - MINIMUM
 - MAXIMUM
10. Sort the summarized results.
11. Generate appropriate plots:
 - Bar chart for categories.
 - Line chart for time-series.
 - Histogram for distributions.
12. Add title, labels, and legend.
13. Save the summary and plots.
14. Validate output files.

END

This workflow provides a complete pipeline from **file input** → **preprocessing** → **grouping** → **aggregation** → **visualization** → **output**.

6. Design a pseudocode algorithm to perform data cleaning and transformation on time-series data. The algorithm should handle missing timestamps, interpolate missing values, remove outliers, and resample data at fixed intervals. Include steps for indexing by time, filtering invalid entries, and ensuring chronological consistency while maintaining efficiency.

Answer:

Input: Time-series dataset containing timestamps and measurements.

Output: Clean and regularly sampled time-series dataset.

Pseudocode

BEGIN

1. Read the time-series dataset.
2. Convert timestamp column into datetime format.
3. Set timestamp as the DataFrame index.
4. Sort records chronologically.
5. Check for duplicate timestamps.

- Remove or combine duplicates.
6. Detect invalid timestamps.
Remove records with invalid timestamps.
 7. Identify missing time intervals.
 8. Resample data at the required fixed interval.
 9. Detect outliers:
Calculate mean and standard deviation
OR calculate IQR.
 10. Remove or replace values identified as outliers.
 11. Interpolate short missing intervals.
 12. Validate:
Check chronological ordering.
Check timestamp frequency.
Check remaining missing values.
Check valid measurement ranges.
 13. Reset index if required.
 14. Save the cleaned time-series data.

END

Resampling and interpolation should be performed using optimized time-series operations. Long missing intervals should not be blindly interpolated because this can create misleading values.

7. Write a detailed pseudocode algorithm to implement feature selection on a dataset. The algorithm should compute correlation between features, remove redundant or irrelevant attributes, and retain significant features. Include steps for normalization, threshold-based filtering, and validation to ensure improved model performance and reduced dimensionality.

Answer:

Input: Dataset containing multiple features.

Output: Reduced dataset containing significant features.

Pseudocode

BEGIN

1. Load the dataset.
2. Separate input features and target variable.
3. Identify numerical features.
4. Handle missing values.

5. Normalize numerical features:
 Apply Min-Max or Z-score normalization.
6. Calculate correlation matrix between features.
7. Define correlation threshold.
8. FOR each pair of features:
 IF absolute correlation > threshold:
 Mark one feature as redundant.
9. Calculate relevance of features with respect to target.
10. Remove irrelevant features using a relevance threshold.
11. Retain significant features.
12. Create reduced feature matrix.
13. Validate:
 Check number of remaining features.
 Check for duplicate features.
 Check for missing values.
14. Train a model using:
 Original features.
 Selected features.
15. Compare model performance.
16. Keep the feature set that provides better
 performance with reduced dimensionality.

END

Correlation helps remove redundant attributes, while threshold-based filtering reduces dimensionality. Validation ensures that important predictive information is not accidentally removed.

8. Develop a pseudocode algorithm to perform data partitioning for machine learning, splitting a dataset into training, validation, and testing sets. Ensure randomization, stratification (if required), and balanced class distribution. Include steps for indexing, shuffling, and validating that no data leakage occurs between partitions.

Answer:

Input: Dataset containing features and target labels.

Output: Training, validation, and testing datasets.

Pseudocode

BEGIN

1. Load the dataset.

2. Separate:
 X = input features
 Y = target labels
3. Remove invalid or duplicate records.
4. Assign a unique index to every record.
5. Shuffle the dataset randomly.
6. IF classification problem:
 Apply stratified splitting
 to preserve class proportions.
7. Split dataset:
 Training set = 70%
 Validation set = 15%
 Testing set = 15%
8. Store the indices of each partition.
9. Check that:
 Training indices $\cap$ Validation indices = empty
 Training indices $\cap$ Testing indices = empty
 Validation indices $\cap$ Testing indices = empty
10. Compare class distributions across partitions.
11. Verify that no record appears in multiple partitions.
12. Fit preprocessing parameters only using
 the training data.
13. Apply the learned preprocessing to validation
 and testing data.
14. Return the three datasets.

END

Using separate indices and checking their intersections prevents data leakage. Stratification is especially important for imbalanced classification datasets.

9. Create a pseudocode algorithm to detect and handle outliers in a dataset using statistical methods such as Z-score or IQR. The algorithm should identify extreme values, decide whether to remove or cap them, and ensure that the resulting dataset maintains consistency and reliability for further analysis.

Answer:

Input: Numerical dataset.

Output: Dataset with appropriately handled outliers.

Pseudocode**

BEGIN

1. Load the numerical dataset.
2. Handle missing values.
3. Select numerical columns.
4. Choose outlier detection method.
5. IF using Z-score:
 - Calculate mean and standard deviation.
 - Calculate:
$$Z = (\text{value} - \text{mean}) / \text{standard_deviation}$$
 - Mark values where $|Z| > \text{threshold}$.
6. ELSE IF using IQR:
 - Calculate Q1 and Q3.
 - $\text{IQR} = Q3 - Q1$.
 - Lower limit = $Q1 - 1.5 \times \text{IQR}$.
 - Upper limit = $Q3 + 1.5 \times \text{IQR}$.
 - Mark values outside these limits.
7. Examine identified outliers.
8. Decide treatment:
 - Remove invalid outliers
 - OR cap extreme values at boundaries
 - OR retain legitimate extreme observations.
9. Validate the cleaned data.
10. Check:
 - Minimum and maximum values.
 - Number of removed records.
 - Remaining missing values.
 - Data distribution.
11. Save the cleaned dataset.

END

The choice between removing and capping depends on whether the outlier represents an error or a legitimate extreme observation. This prevents valid information from being unnecessarily lost.

10. Write a pseudocode algorithm to implement data aggregation and summarization across multiple categories. The algorithm should group data based on specified attributes, compute aggregated metrics (sum, average, count), and generate a

summarized report. Include steps for sorting, indexing, and validating the correctness of aggregated results.

Answer:

Input: Dataset and grouping attributes.

Output: Aggregated summary report.

Pseudocode

BEGIN

1. Load the dataset.
2. Validate required columns.
3. Handle missing and invalid values.
4. Select grouping attributes.
5. Group records according to the selected attributes.
6. Calculate:
 - Total = SUM(value)
 - Average = MEAN(value)
 - Count = COUNT(records)
7. Store results in a summary DataFrame.
8. Sort the summary by the required metric.
9. Reset the summary index.
10. Validate:
 - Compare total count with original valid records.
 - Check group names.
 - Check aggregate values.
 - Check for missing groups.
11. Generate a formatted summary report.
12. Save the report.

END

Grouping allows data to be analyzed category-wise. Sorting makes the report easier to interpret, while validation ensures that aggregation has not accidentally omitted or duplicated records.

11. Design a pseudocode algorithm to perform data normalization and scaling on a dataset. The algorithm should support techniques such as Min-Max scaling and Z-score normalization. Include steps for identifying numerical columns, applying transformations, handling missing values, and validating that all features are scaled correctly within specified ranges.

Answer:

Input: Dataset containing numerical and categorical columns.

Output: Scaled numerical dataset.

Pseudocode

BEGIN

1. Load dataset.
2. Identify numerical columns.
3. Check for missing values.
4. Handle missing numerical values:
 Replace with mean or median.
5. Select normalization method.
6. IF Min-Max scaling:
 Find minimum and maximum for each feature.
 Apply:

$$X_{\text{scaled}} = \frac{(X - X_{\text{min}})}{(X_{\text{max}} - X_{\text{min}})}$$
7. IF Z-score normalization:
 Calculate mean and standard deviation.
 Apply:

$$X_{\text{scaled}} = (X - \text{mean}) / \text{standard_deviation}$$
8. Keep categorical columns unchanged
 unless encoding is required.
9. Validate scaled data:
 For Min-Max:
 Check values are approximately between 0 and 1.
 For Z-score:
 Check mean is approximately 0
 and standard deviation is approximately 1.
10. Save the transformed dataset.

END

Scaling should be fitted using the training data in machine-learning applications to prevent data leakage.

12. Write a pseudocode algorithm to implement dimensionality reduction using Principal Component Analysis (PCA). The algorithm should include steps for standardizing the dataset, computing covariance matrix, extracting eigenvalues and eigenvectors, and selecting principal components. Ensure efficient matrix operations and validation of reduced dimensions.

Answer:

Input: Numerical dataset with multiple features.

Output: Dataset transformed into principal components.

Pseudocode

BEGIN

1. Load the numerical dataset.
2. Handle missing values.
3. Standardize all features:
 - Calculate feature mean.
 - Calculate feature standard deviation.
 - Apply Z-score normalization.
4. Compute covariance matrix.
5. Calculate eigenvalues and eigenvectors of the covariance matrix.
6. Sort eigenvalues in descending order.
7. Rearrange eigenvectors according to descending eigenvalues.
8. Calculate explained variance ratio.
9. Select the first K principal components based on required explained variance.
10. Create projection matrix using selected eigenvectors.
11. Transform the standardized data:
$$\text{Reduced_Data} = \text{Standardized_Data} \times \text{Projection_Matrix}$$
12. Validate:
 - Check original dimensions.
 - Check reduced dimensions.
 - Check explained variance.
13. Return the reduced dataset.

END

Efficient matrix operations should be used instead of element-by-element loops. The reduced dataset must have fewer dimensions while retaining an acceptable amount of the original variance.

13. Develop a pseudocode algorithm to perform data merging and conflict resolution from multiple sources. The algorithm should detect conflicting attribute values, apply

resolution strategies (e.g., priority rules or averaging), and ensure consistency across the final dataset. Include steps for indexing, validation, and handling missing or inconsistent records.

Answer:

Input: Multiple datasets containing common records.

Output: Unified dataset with resolved conflicts.

Pseudocode

BEGIN

1. Load all source datasets.
2. Standardize:
 - Column names.
 - Data types.
 - Formats.
3. Identify the common unique key.
4. Set or align indexes using the common key.
5. Merge the datasets using the common key.
6. Detect records with conflicting values.
7. FOR each conflicting attribute:
 - IF one source has higher priority:
 - Select value from higher-priority source.
 - ELSE IF values are numerical:
 - Calculate average when appropriate.
 - ELSE:
 - Select the most reliable or frequent value.
8. Handle missing values:
 - Fill from another source when valid.
 - Otherwise retain as missing.
9. Remove duplicate records.
10. Validate:
 - Check unique keys.
 - Check missing values.
 - Check data types.
 - Check unresolved conflicts.
11. Sort and reset the final index.
12. Save the unified dataset.

END

Conflict resolution rules should be defined before merging. This ensures consistent treatment of conflicting values and prevents arbitrary data changes.

14. Create a pseudocode algorithm to implement rolling window computations on a time-series dataset. The algorithm should calculate moving average and rolling variance using efficient indexing and windowing techniques. Include steps for handling boundary conditions, missing values, and optimizing performance for large datasets.

Answer:

Input: Time-series dataset and window size.

Output: Moving average and rolling variance.

Pseudocode

BEGIN

1. Load the time-series dataset.
2. Convert timestamp column to datetime.
3. Set timestamp as index.
4. Sort the dataset chronologically.
5. Define window size K.
6. Handle missing values according to requirements.
7. FOR each rolling window:
 - Select K consecutive observations.
 - IF sufficient valid observations exist:
 - Calculate:
 - Moving_Average = mean(window)
 - Rolling_Variance = variance(window)
 - ELSE:
 - Assign missing value.
8. Store calculated values in new columns.
9. Validate boundary conditions:
 - Check first K-1 observations.
 - Ensure insufficient windows are handled correctly.
10. Check alignment between timestamps and calculated statistics.
11. Optimize using vectorized rolling operations.
12. Return the final time-series DataFrame.

END

For large datasets, built-in rolling functions should be preferred because they are optimized compared with manual loops. Missing values and the minimum number of observations required should be clearly defined.

15. Write a pseudocode algorithm to perform data validation and integrity checks on a dataset before analysis. The algorithm should verify data types, detect missing or invalid entries, enforce constraints (such as value ranges), and generate a validation report. Include steps for filtering errors, logging issues, and ensuring the dataset is ready for further processing.

Answer:

Input: Raw dataset and predefined validation rules.

Output: Validated dataset and validation report.

Pseudocode

BEGIN

1. Load the dataset.
2. Create an empty validation report.
3. Check required columns:
 IF any required column is missing:
 Log the error.
4. Check data types:
 Compare actual types with expected types.
 Log inconsistent types.
5. Check missing values:
 Count missing values in each column.
 Log columns containing missing values.
6. Check duplicate records:
 Identify duplicate rows.
 Log duplicate count.
7. Check numerical constraints:
 FOR each numerical column:
 IF value is outside valid range:
 Mark the record as invalid.
 Log the violation.
8. Check categorical constraints:
 Compare values with allowed categories.
 Log invalid categories.
9. Filter invalid records when required.
10. Generate validation summary:
 Total records.
 Valid records.

Invalid records.
Missing values.
Duplicate records.
Constraint violations.

11. IF critical errors exist:
 Stop further processing.
 Return validation report.

12. ELSE:
 Confirm dataset is ready for analysis.

13. Save the validation report.

14. Return validated dataset.

END

A validation report provides a clear record of all detected problems. Logging errors before removing or correcting them improves traceability and data integrity. The dataset should proceed to analysis only after all critical constraints have been satisfied.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	25	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand